

Presentación resumen de Java

Rodrigo Blanco Betes. Alfa Formación

Enero del 2026

- 1 Introducción a Java
- 2 Tipos primitivos y objetos. API de Java
- 3 Estructuras de Control de Flujo
- 4 Arrays en Java
- 5 Programación Orientada a Objetos en Java
- 6 Clases y Objetos en Java
- 7 Constructores
- 8 Métodos
- 9 Uso de Modificadores de Acceso

Java es un lenguaje de programación de propósito general ampliamente utilizado en:

- Aplicaciones de escritorio
- Aplicaciones web
- Sistemas empresariales
- Dispositivos móviles

Se caracteriza principalmente por su:

- Portabilidad
- Seguridad
- Robustez

Java fue desarrollado a principios de la década de 1990 por un equipo liderado por James Gosling en Sun Microsystems.

- Proyecto original: *Green Project*
- Nombre inicial: *Oak*
- Cambio de nombre a *Java* en 1995
- Lema: *"Write Once, Run Anywhere"*

En 2010, Sun Microsystems fue adquirida por Oracle Corporation, que actualmente mantiene y evoluciona el lenguaje.

Principales Características de Java

- **Portabilidad:** Compilación a *bytecode* ejecutable en cualquier sistema con JVM.
- **Orientación a objetos:** Facilita la reutilización, mantenimiento y organización del código.
- **Seguridad:** Verificación de bytecode, gestión automática de memoria y modelo de permisos.
- **Gestión automática de memoria:** Uso de recolector de basura.
- **Multihilo:** Soporte para programación concurrente mediante hilos.

Java es un lenguaje maduro y ampliamente utilizado que ha demostrado su utilidad a lo largo del tiempo.

- Funciona en múltiples plataformas
- Es seguro y robusto
- Sigue siendo una opción clave en el desarrollo de software

Tipos primitivos y objetos. API de Java

El lenguaje de programación Java ofrece un conjunto sólido de herramientas fundamentales para el desarrollo de aplicaciones. Entre ellas se encuentran los tipos primitivos, los objetos proporcionados por su API estándar y los instrumentos de control de flujo, que permiten definir la lógica y el comportamiento de los programas.

Los tipos primitivos son los tipos de datos básicos que Java proporciona para almacenar valores simples. Estos no son objetos y ocupan un espacio fijo en memoria.

Tipos primitivos

Los tipos primitivos son:

- **byte**: almacena números enteros pequeños.
- **short**: almacena números enteros de tamaño reducido.
- **int**: tipo entero más utilizado.
- **long**: almacena números enteros grandes.
- **float**: almacena números decimales de precisión simple.
- **double**: almacena números decimales de doble precisión.
- **char**: representa un carácter Unicode.
- **boolean**: almacena valores lógicos (true o false).

El uso de tipos primitivos permite una mayor eficiencia en el uso de memoria y un mejor rendimiento del programa.

Introducción a la API de Java

Java incluye una extensa biblioteca estándar conocida como la API de Java, la cual proporciona clases y objetos reutilizables que facilitan el desarrollo de software.

Algunos ejemplos importantes de objetos de la API son:

- **String**: permite manipular cadenas de texto.
- **Scanner**: se utiliza para la entrada de datos desde el teclado.
- **Math**: ofrece métodos para realizar operaciones matemáticas.
- **ArrayList**: permite almacenar colecciones dinámicas de datos.
- **Date**: se utiliza para manejar fechas y tiempos.

Estos objetos permiten al programador evitar la implementación desde cero de funcionalidades comunes, aumentando la productividad y reduciendo errores.

Estructuras de Control de Flujo

Los estructuras de control de flujo permiten decidir el orden de ejecución de las instrucciones dentro de un programa Java.

Por ejemplo, las estructuras condicionales permiten ejecutar bloques de código según se cumpla o no una condición.

- **if**
- **if-else**
- **switch**

Las estructuras repetitivas permiten ejecutar un bloque de código varias veces.

- **for**
- **while**
- **do-while**

Estas instrucciones modifican el flujo normal del programa.

- **break**: finaliza un bucle o un `switch`.
- **continue**: salta a la siguiente iteración de un bucle.
- **return**: devuelve un valor y finaliza la ejecución de un método.

Introducción a los arrays

Un array es una estructura de datos que permite almacenar múltiples valores del mismo tipo bajo un mismo nombre.

Características principales:

- Todos los elementos son del mismo tipo
- Se accede mediante índices que comienzan en cero
- El tamaño es fijo

Ejemplos de declaración e inicialización:

```
int[] numeros = new int[5];
```

```
int[] numeros = {1, 2, 3, 4, 5};
```

Uso de Arrays

Acceso a los elementos mediante su índice:

```
int valor = numeros[0];
```

Recorrido de un array con un bucle for:

```
for (int i = 0; i < numeros.length; i++) {  
    System.out.println(numeros[i]);  
}
```

```
for (int numero: numeros) {  
    System.out.println(numeros[i]);  
}
```

Ventajas y limitaciones de los arrays

Ventajas y limitaciones:

- Acceso rápido y eficiente a los datos
- Uso óptimo de la memoria
- Tamaño fijo que no puede modificarse

La Clase JOptionPane

La clase `JOptionPane` pertenece al paquete `javax.swing` y permite crear ventanas de diálogo gráficas de forma sencilla, sin interfaces complejas.

Ejemplos de uso:

- Mostrar mensajes
- Solicitar datos al usuario
- Confirmar acciones

```
JOptionPane.showMessageDialog(null, "Hola mundo");  
  
String nombre =  
JOptionPane.showInputDialog("Introduce tu nombre");  
  
int opcion =  
JOptionPane.showConfirmDialog(null, "¿Deseas continuar?");
```

Por su simplicidad, es muy recomendable para programas educativos e interacciones sencillas con el usuario.

Programación Orientada a Objetos en Java

Java implementa la Programación Orientada a Objetos (POO) mediante el uso de clases y objetos, permitiendo modelar problemas complejos de forma estructurada.

Los cuatro pilares fundamentales de la POO son:

- Encapsulación
- Abstracción
- Herencia
- Polimorfismo

- **Encapsulación:** oculta los detalles internos de una clase y expone solo lo necesario mediante modificadores de acceso.
- **Abstracción:** representa las características esenciales de un objeto, ignorando detalles irrelevantes.
- **Herencia:** permite que una clase herede atributos y métodos de otra.
- **Polimorfismo:** un mismo método puede comportarse de forma distinta según el objeto que lo invoque.

Una clase es una plantilla que define las propiedades y comportamientos de un objeto.

Un objeto es una instancia concreta de una clase creada a partir de dicha plantilla.

Estructura Básica de una Clase

Una clase en Java se define utilizando la palabra reservada `class`.

```
public class Persona {  
    String nombre;  
    int edad;  
}
```

Creación de Objetos

Para crear un objeto a partir de una clase se utiliza el operador `new`.

```
Persona p1 = new Persona();
```

Los constructores son métodos especiales que se ejecutan al crear un objeto.

- Tienen el mismo nombre que la clase
- Se utilizan para inicializar los atributos

```
public Persona(String nombre, int edad) {  
    this.nombre = nombre;  
    this.edad = edad;  
}
```

Los métodos definen el comportamiento de los objetos y permiten realizar acciones o devolver información.

Ejemplo de método en Java:

```
public void mostrarDatos() {  
    System.out.println(nombre + " - " + edad);  
}
```

Modificadores de Acceso

Java utiliza modificadores de acceso para controlar la visibilidad de atributos y métodos:

- `public`: accesible desde cualquier clase
- `private`: accesible solo desde la misma clase
- `protected`: accesible desde clases del mismo paquete o subclases